KRITI 2026 • IIT GUWAHATI

BOT DESIGN CHALLENGE

Autonomous Rover for Navigation and Real-Time Image Recognition

Team ID	**2613**
Problem	Keus Bot Design Challenge
Sponsor	Keus Smart Home
Stage	Kriti 2026 — Mid Prep

TECHNOLOGY STACK

Arduino Uno ESP32 DevModule Raspberry Pi 5 YOLOv8-nano Flask-SocketIO

Executive Summary

This report presents the complete design, development, and implementation of a fully autonomous ground rover for the Kriti 2026 Bot Design Challenge, sponsored by Keus Smart Home. The system is engineered as a **three-tier embedded architecture**.

Tier 1 — Arduino Uno handles real-time navigation via an eight-sensor IR array (five for line following, three for obstacle detection) and dual-mode obstacle avoidance (side IRs and front IR). **Tier 2 — ESP32 DevModule** runs a FreeRTOS dual-core perception engine managing MPU6050-based IMU-guided turning, ultrasonic radar sweep, and camera servo positioning. **Tier 3 — Raspberry Pi 5** executes a five-model machine learning ensemble comprising YOLOv8-nano, MobileNetV3-Small embeddings, dlib face recognition, pyzbar QR decoding, and Tesseract OCR, with results streamed in real time to a Flask-SocketIO web dashboard.

The rover navigates a 15 ft × 20 ft floor-marked arena, follows a symmetric two-track path with junction-aware state tracking, dynamically avoids a 250 mm cube obstacle using gyroscope-guided 90° turns, and identifies all 16 A4-format image cards. The live operator dashboard displays camera feed, identified targets against a running 0/16 counter, timestamped logs, and a competition timer — entirely over Wi-Fi, with no cloud dependency. All computation is fully onboard.

Contents

1 Problem Statement Overview

1.1 Challenge Introduction

The Kriti 2026 Bot Design Challenge requires teams to build a **fully autonomous rover** that navigates a predefined arena while performing real-time image detection and identification — without any remote control or cloud processing.

The rover must traverse a symmetric floor-marked path from point A to point B (and back in the second run), identify 16 image cards placed throughout the arena, and transmit all recognition results wirelessly to an operator laptop via a live web interface. Each team receives two attempts of up to 5 minutes; the best run is scored.

1.2 Arena Layout

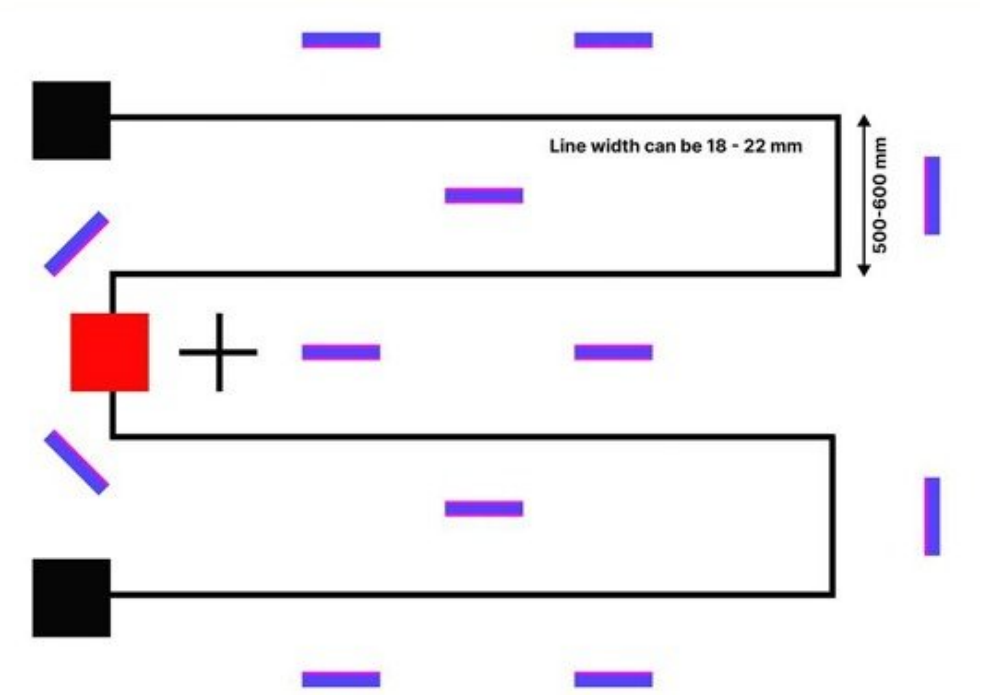

Figure 1. Actual arena layout (15 ft × 20 ft) — Kriti 2026 Bot Design Challenge. Black path with 18–22 mm line width; red obstacle (250 mm cube); purple image cards at periphery, centre corridor, and at 45° near obstacle; track segments of 500–600 mm width.

1.3 Key Constraints

Table 1. Technical and operational constraints

Category	Constraint	Details
Technical	Max dimensions	25 cm × 25 cm × 25 cm at all times
	Max weight	3 kg
	Run duration	5 minutes per run
	Power source	Onboard battery only; no external supply
Off-path	Per instance	Must rejoin path within 10 seconds
	Total per run	$\leq$ 30 seconds off-path per run
Hardware	Controllers	Arduino, ESP32/ESP8266, Raspberry Pi only
	Forbidden	Jetson Nano, external SBCs, cloud/laptop compute
	Peripherals	Cameras, sensors, motor drivers permitted
Task	Image cards	Exactly 16; any identification order
	End condition	Reach end position after all identifications

1.4 Evaluation Parameters

Table 2. Competition evaluation parameters (9 total)

#	**Parameter**	**Key Metrics**
1	Navigation & Mobility	Path accuracy, junction precision, deviation recovery
2	Obstacle Detection & Avoidance	Real-time detection, smooth deviation, re-entry
3	Vision & Image Recognition	Detection reliability, orientation handling, accuracy
4	Communication & Data Display	Wi-Fi stability, real-time display, formatting
5	System Autonomy & Execution	Sequential execution, error recovery, termination
6	Hardware Design & Integration	Stability, sensor placement, size compliance
7	Software Architecture	Modularity, documentation, pipeline reliability
8	Presentation & Documentation	Technical clarity, architecture, video quality
9	Live Hardware Demonstration	Task completion, timing reliability, build quality

2 System Architecture

2.1 Three-Tier Architecture

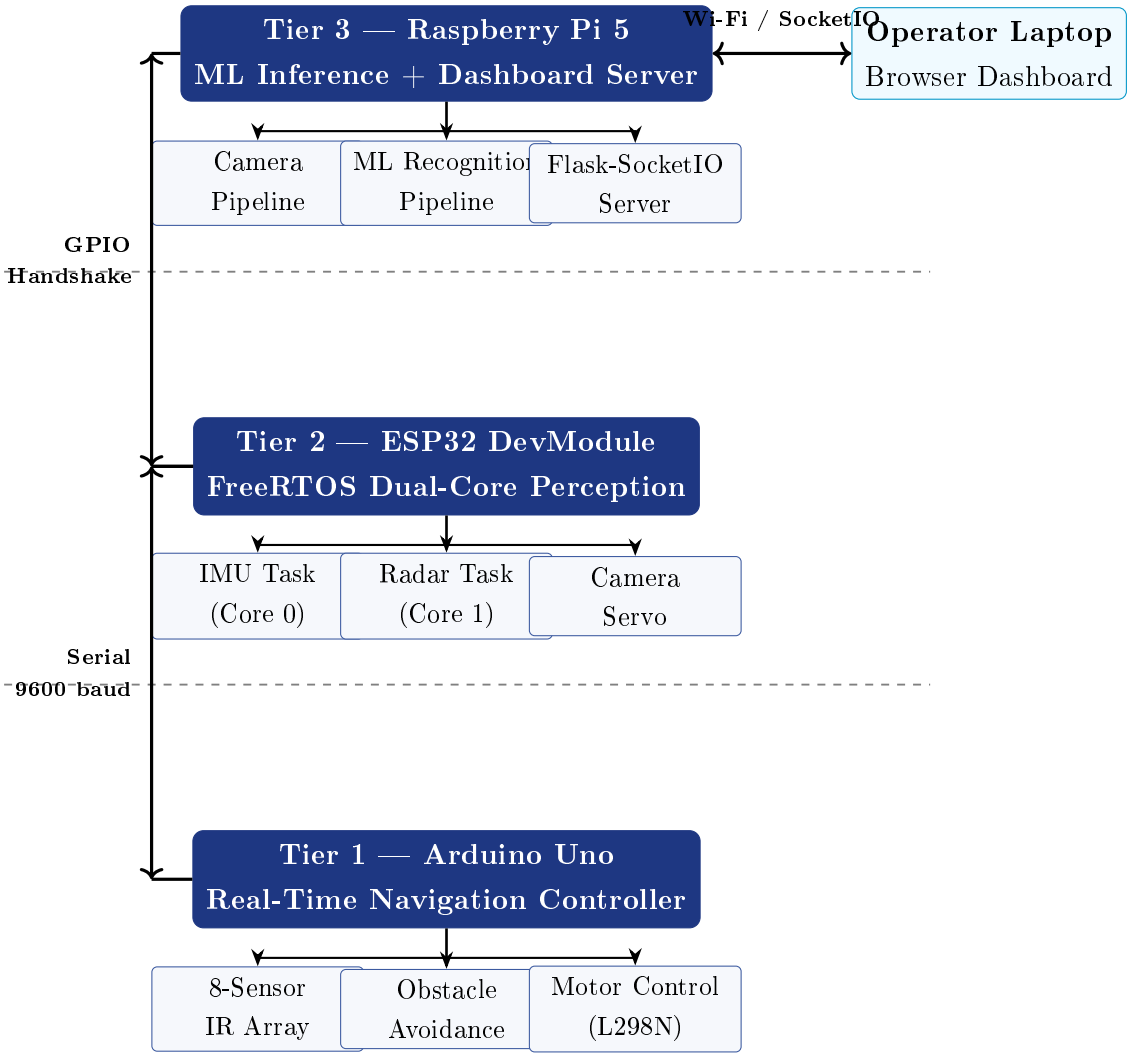

Figure 2. Three-tier system architecture with inter-board communication protocols.

2.2 Communication Protocol Summary

Table 3. Inter-system communication channels

Link	From → To	Protocol	Data / Purpose
A	Arduino → ESP32	GPIO A2 (digital)	IMU_Reset: LOW→HIGH triggers yaw zero
B	Arduino → ESP32	GPIO A4 (digital)	RadarScan: HIGH triggers full sweep
C	ESP32 → Arduino	UART2 @ 9600 baud	`IMU:angle\n`; `Obstacle Detected`; `DONE`
D	Arduino ↔ ESP32	SoftwareSerial 9600	Shared bus for all serial messages
E	ESP32 → RPi	GPIO23 (`toPI`)	HIGH pulse: trigger image capture
F	RPi → ESP32	GPIO34 (`fromPI`)	HIGH acknowledgement: capture done
G	RPi → Dashboard	Wi-Fi / SocketIO	Base64 JPEG frames; recognition results
H	Dashboard → Browser	HTTP + WebSocket	Live UI updates, logs, target grid

2.3 Subsystem Responsibilities

Table 4. Functional responsibilities by controller

Controller	Responsibilities
Arduino Uno	5-sensor priority-cascade line following; `lastRight` junction state tracking; side obstacle avoidance (side IRs obsL, obsR); front obstacle avoidance (serial from ESP32); IMU-guided 90° turn requests; full radar scan trigger at every line re-entry; motor PWM via L298N; 3-second avoidance cooldown.
ESP32 DevModule	**Core 0:** MPU6050 gyroscope integration with startup bias calibration and complementary filter; streams angle to Arduino at 50 Hz. **Core 1:** Ultrasonic distance measurement (front/left/right, 40 cm threshold); radar servo sweep 30°–150°; camera servo positioning; GPIO handshake with Pi; full sweep on RadarScan.
Raspberry Pi 5	Frame acquisition (Pi Camera Module 3 / OpenCV fallback); Laplacian sharpness-based frame selection from 20-frame rolling buffer; A4 sheet detection and perspective correction; 5-model ML inference pipeline; Flask-SocketIO server; live video streaming at ≈10 FPS; timestamped recognition log.

3 Hardware Design

3.1 Chassis and Mechanical Design

The rover uses a two-layer laser-cut 3 mm acrylic chassis with brass standoffs separating the layers for double-deck component mounting. The chassis fits within the 25 cm × 25 cm × 25 cm competition envelope at all times.

- **Bottom layer:** DC gear motors (×2) with rubber wheels; array of 5 IR sensors (bottom-facing, line following); 12 V LiPo battery compartment (motor power supply).
- **Top layer:** Arduino Uno; ESP32 DevModule; L298N motor driver; mini breadboards; MPU6050 IMU module; wiring harness; power bank (RPi supply) and Li-ion 18650 cell (ESP32 supply) secured to top platform.
- **Servo mounts:** Radar servo (30°–150° sweep with front ultrasonic) and Pi Camera Module 3 pan servo mounted centrally on the top layer.
- **Front bracket:** Front-facing HC-SR04 ultrasonic sensor and front IR sensor for obstacle detection, mounted at obstacle-detection height.
- **Side brackets:** Left and right HC-SR04 ultrasonic sensors and side IR sensors (obsL, obsR) mounted on left and right flanks at obstacle-detection height.

3.2 Pin Mapping Tables

3.2.1 Arduino Uno

Table 5. Arduino Uno pin mapping

Pin	Signal	Connected To	Purpose
D4	R_SENSOR	IR sensor (right)	Line following
D8	M_SENSOR	IR sensor (centre)	Line following
D9	LL_SENSOR	IR sensor (far left)	Sharp left / junction detect
D13	L_SENSOR	IR sensor (left)	Soft left correction
A3	RR_SENSOR	IR sensor (far right)	Sharp right / junction detect
A0	obsL	IR sensor (left side)	Side obstacle detection
A1	obsR	IR sensor (right side)	Side obstacle detection
A2	IMU_Reset	ESP32 GPIO4	Trigger IMU yaw zero
A4	RadarScan	ESP32 GPIO33	Trigger full radar sweep
D2	SoftSerial RX	ESP32 GPIO17 (TX)	Receive serial from ESP32
D3	SoftSerial TX	ESP32 GPIO16 (RX)	Transmit serial to ESP32
D5	MOTOR1_EN	L298N ENA	Motor 1 PWM speed
D6	MOTOR1_IN1	L298N IN1	Motor 1 direction
D7	MOTOR1_IN2	L298N IN2	Motor 1 direction
D10	MOTOR2_EN	L298N ENB	Motor 2 PWM speed
D11	MOTOR2_IN2	L298N IN4	Motor 2 direction
D12	MOTOR2_IN1	L298N IN3	Motor 2 direction

3.2.2 ESP32 DevModule

Table 6. ESP32 DevModule pin mapping

GPIO	Signal	Connected To	Purpose
4	TRIGGER_PIN	Arduino A2	IMU reset trigger input
5	TRIG (front)	HC-SR04 front	Front ultrasonic trigger
13	SERVOC_PIN	Camera servo signal	Camera pan servo PWM
14	IR_SENSOR	Front IR sensor	Front obstacle detection
16	UART2 RX	Arduino D3	Serial receive from Arduino
17	UART2 TX	Arduino D2	Serial transmit to Arduino
18	ECHO (front)	HC-SR04 front	Front ultrasonic echo
19	SERVO_PIN	Radar servo signal	Radar sweep servo PWM
23	toPI	RPi GPIO (input)	Signal Pi to capture image
25	TRIGL	HC-SR04 left	Left ultrasonic trigger
26	ECHOL	HC-SR04 left	Left ultrasonic echo
27	TRIGR	HC-SR04 right	Right ultrasonic trigger
32	ECHOR	HC-SR04 right	Right ultrasonic echo
33	TRIGGERC_PIN	Arduino A4 (RadarScan)	Full radar sweep trigger (input from Arduino)
34	fromPI	RPi GPIO (output)	Acknowledgement from Pi

3.3 Power Architecture

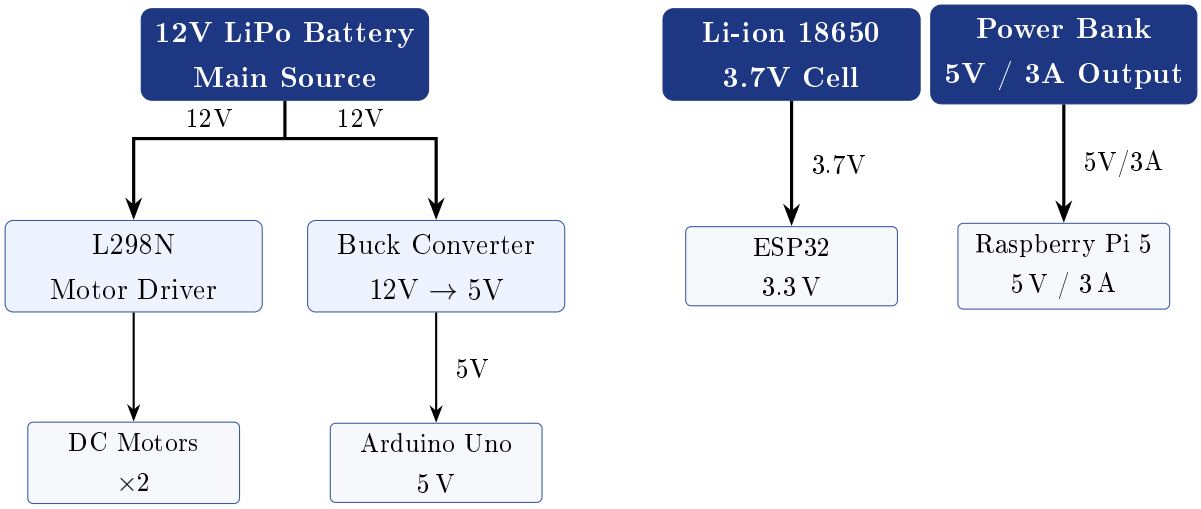

Figure 3. Power distribution. LiPo drives motors (L298N) and Arduino (via buck converter). Li-ion 18650 powers the ESP32 independently. Dedicated power bank supplies the Raspberry Pi, isolating it from motor switching noise.

3.4 Bill of Materials

Table 7. Complete Bill of Materials — prices in Indian Rupees

#	Component	Qty	Unit (Rs.)	Total (Rs.)
1	Raspberry Pi 5 (16 GB)	1	9,500	9,500
2	Arduino Uno R3	1	550	550
3	ESP32 DevModule V1 (38-pin)	1	450	450
4	Raspberry Pi Camera Module 3	1	3,200	3,200
5	MPU6050 IMU Module	1	120	120
6	L298N Motor Driver Module	1	180	180
7	DC Gear Motor with Rubber Wheel	2	350	700
8	SG90 Servo Motor (radar sweep)	1	140	140
9	MG90S Metal Gear Servo (camera mount)	1	220	220
10	HC-SR04 Ultrasonic Sensor	3	80	240
11	IR Sensor Module (TCRT5000-based)	8	50	400
12	Acrylic Sheet 3 mm (laser-cut chassis)	1	600	600
13	12 V LiPo Battery (1300–1800 mAh)	1	1,800	1,800
14	Li-ion 18650 Cell (3.7 V, ESP32 supply)	1	250	250
15	Power Bank (10,000 mAh, 5 V/3 A, RPi supply)	1	900	900
16	Buck Converter Module (LM2596)	1	80	80
17	Perf / Dot Board	2	60	120
18	Mini Breadboard	2	60	120
19	Brass Standoffs + Screws Assortment	1	150	150
20	Jumper Wire Sets (M-M, M-F, F-F)	3	80	240
21	Misc (zip ties, hot glue, tape, headers)	—	—	200
			Grand Total	**20,410**

4 Navigation and Mobility

4.1 Line Following Algorithm

Five downward-facing IR sensors are arranged **LL** – **L** – **M** – **R** – **RR** across the path width. The Arduino evaluates them each loop iteration using a strict priority cascade: the first matching condition determines the motor command for that cycle.

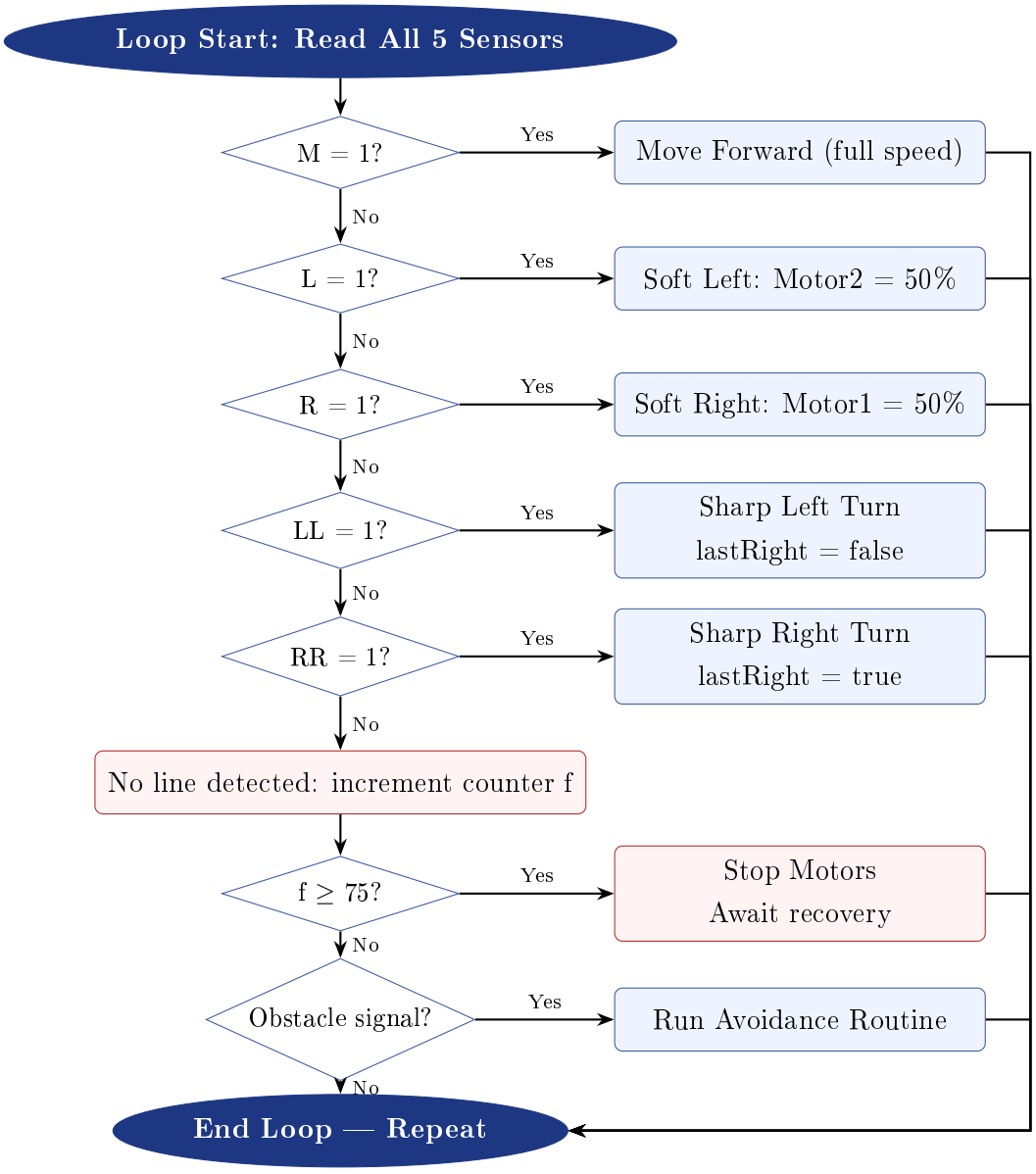

Figure 4. Line following priority cascade with lost-line handler and obstacle check.

The `lastRight` flag is updated at LL/RR junctions. It stores the most recent sharp-turn direction and drives all subsequent obstacle avoidance logic to ensure correct side-

selection for path re-entry.

4.2 Obstacle Avoidance — Side IR Triggered

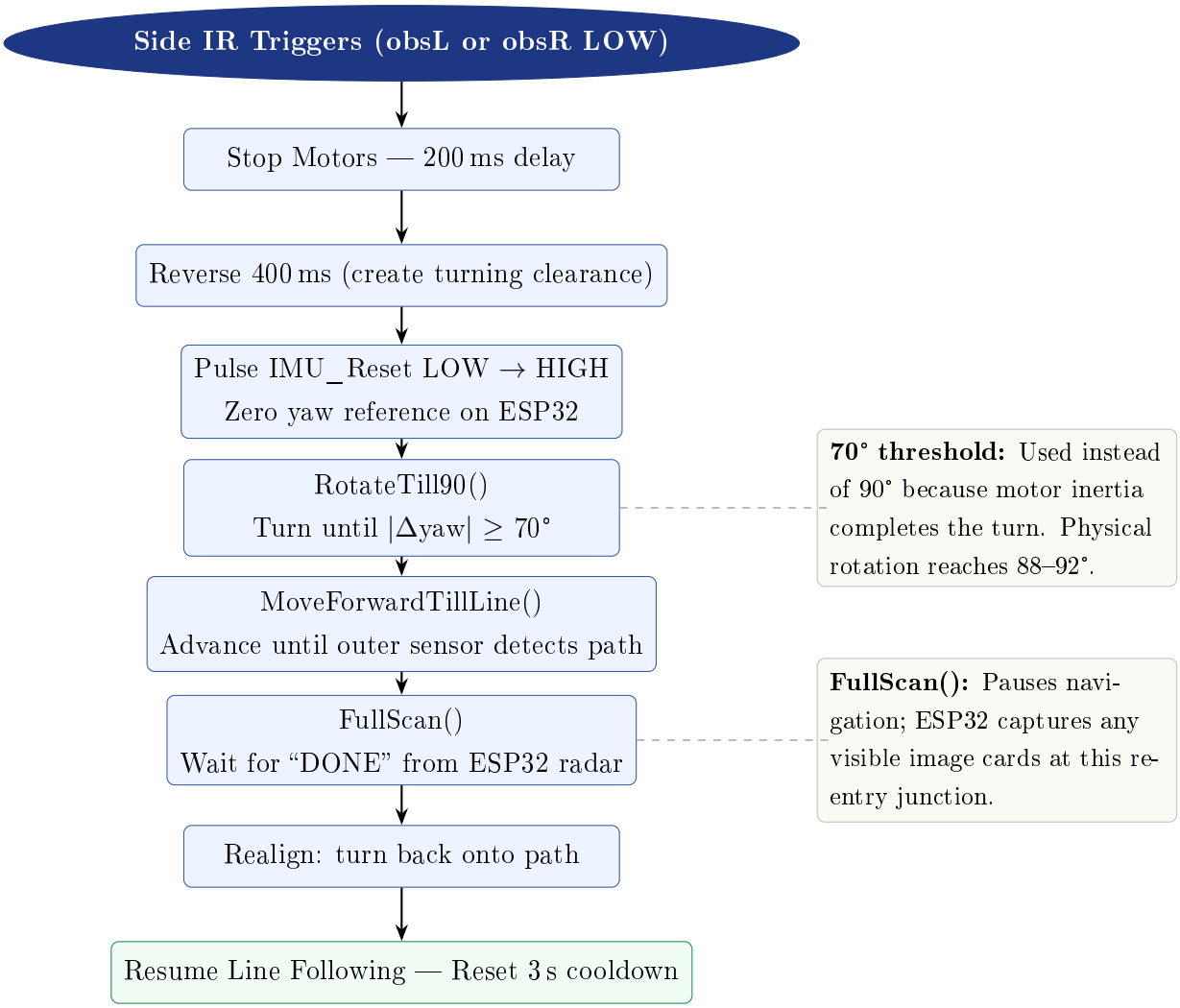

Figure 5. Side obstacle avoidance routine (`obsAvoid`).

4.3 Obstacle Avoidance — Front IR Triggered

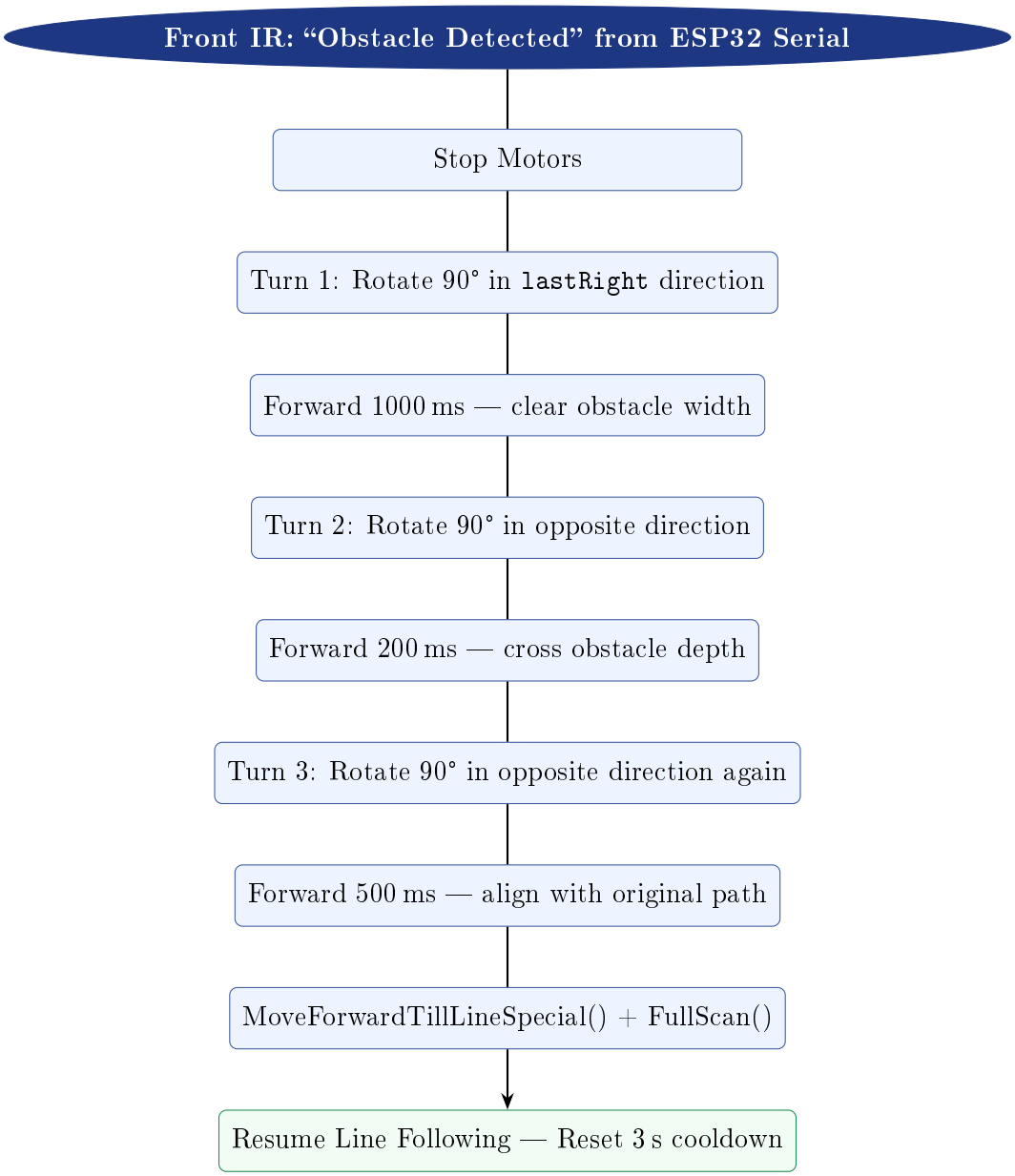

Figure 6. Front obstacle three-turn U-bypass routine (`obsAvoidFront`).

4.4 IMU-Guided Turning

Precise 90° turns use the MPU6050 gyroscope on the ESP32:

1. Arduino pulses `IMU_Reset` (A2): LOW→HIGH.
2. ESP32 Core 0 zeroes the yaw reference (`resetAll`).

3. Arduino reads initial yaw, then drives motors until $|\Delta\text{yaw}| \geq 70$.
4. Motor inertia completes the remaining $\approx 20^\circ$, producing 88–92° physical rotation.

The complementary filter reduces drift: $\theta_{\text{smooth}}[n] = 0.9\,\theta_{\text{smooth}}[n-1] + 0.1\,\theta_{\text{raw}}[n]$

Startup bias: $b_z = \frac{1}{1000}\sum_{i=1}^{1000} \frac{g_z[i]}{131.0}$ (131 LSB per °/s at 250 °/s range)

5 Perception System

5.1 ESP32 Radar System

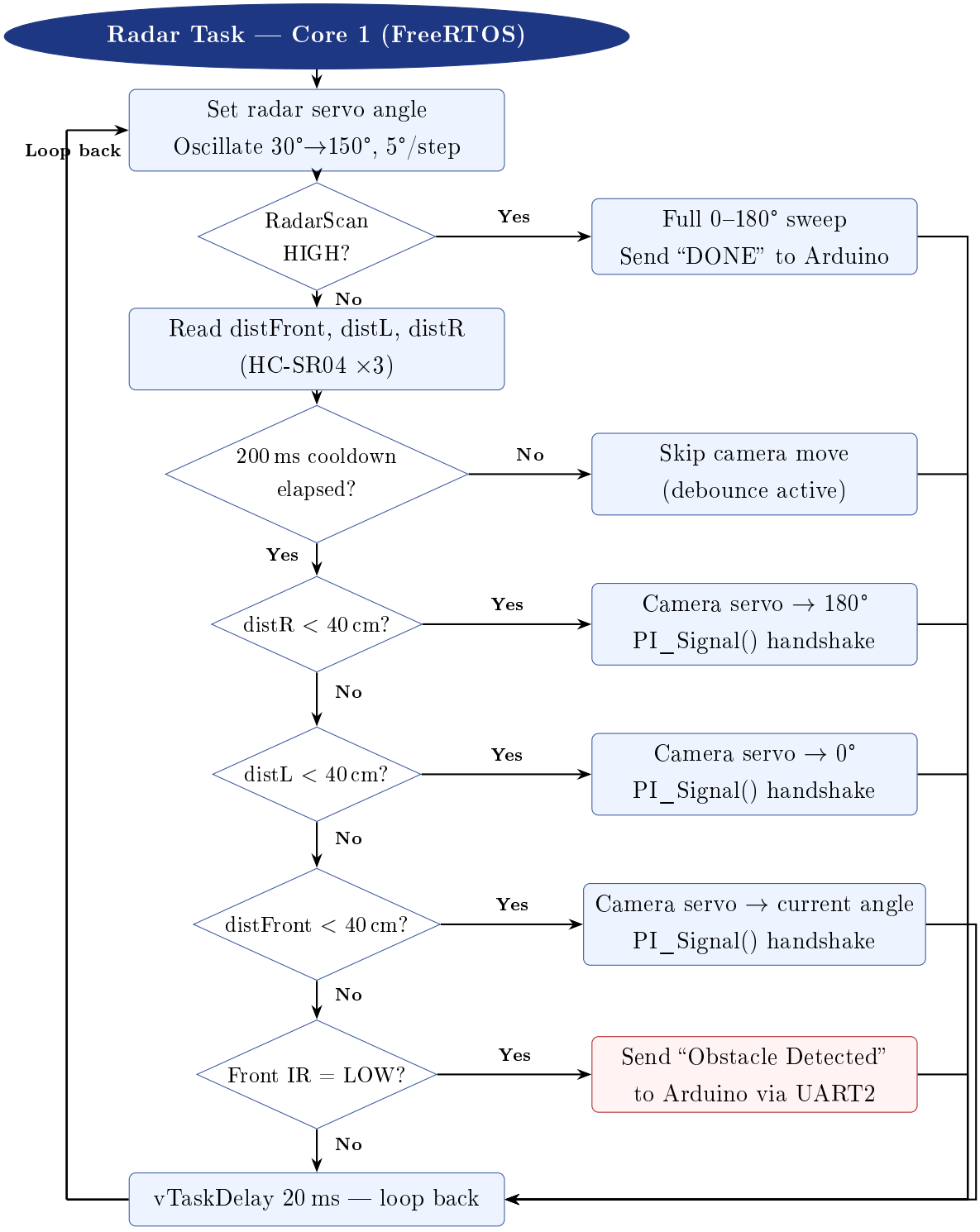

Figure 7. ESP32 Core 1 radar task flowchart.

5.2 Raspberry Pi Camera Pipeline

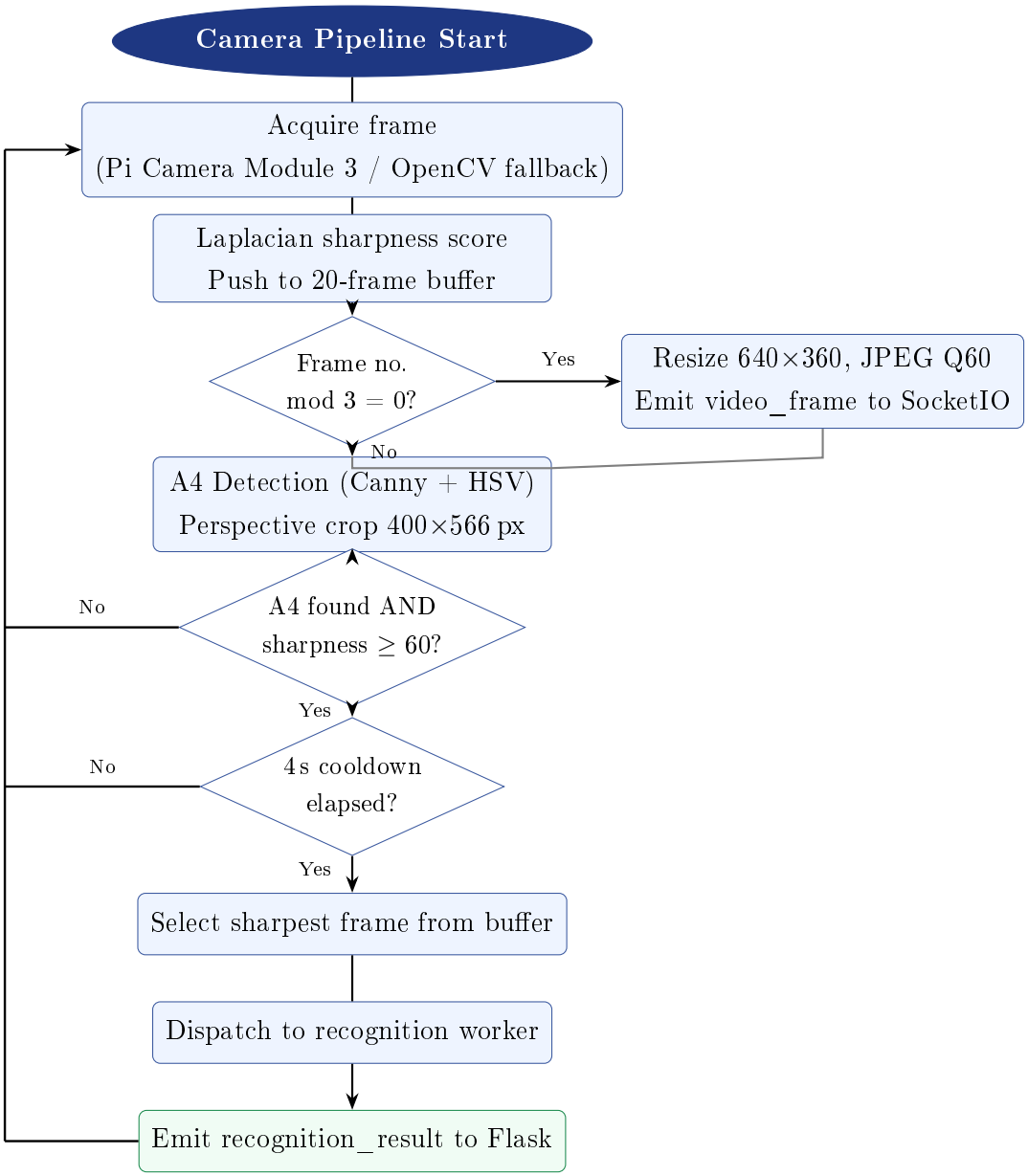

Figure 8. Raspberry Pi camera pipeline (camera_pipeline.py).

5.3 A4 Detection and Perspective Correction

Two parallel detection methods run on every frame:

1. **Canny method:** Gaussian blur → Canny edges (40/130) → dilation → contour find.
2. **HSV white method:** Convert HSV → threshold (H:0–180, S:0–50, V:140–255) → morphological close → contour find.

Both apply: 4-corner polygon, area 10–92%, aspect ratio 1.1–2.0, mean brightness ≥140.

On success, `warpPerspective` produces a standardised **400×566 px** crop.

6 Image Recognition Pipeline

6.1 Pipeline Architecture

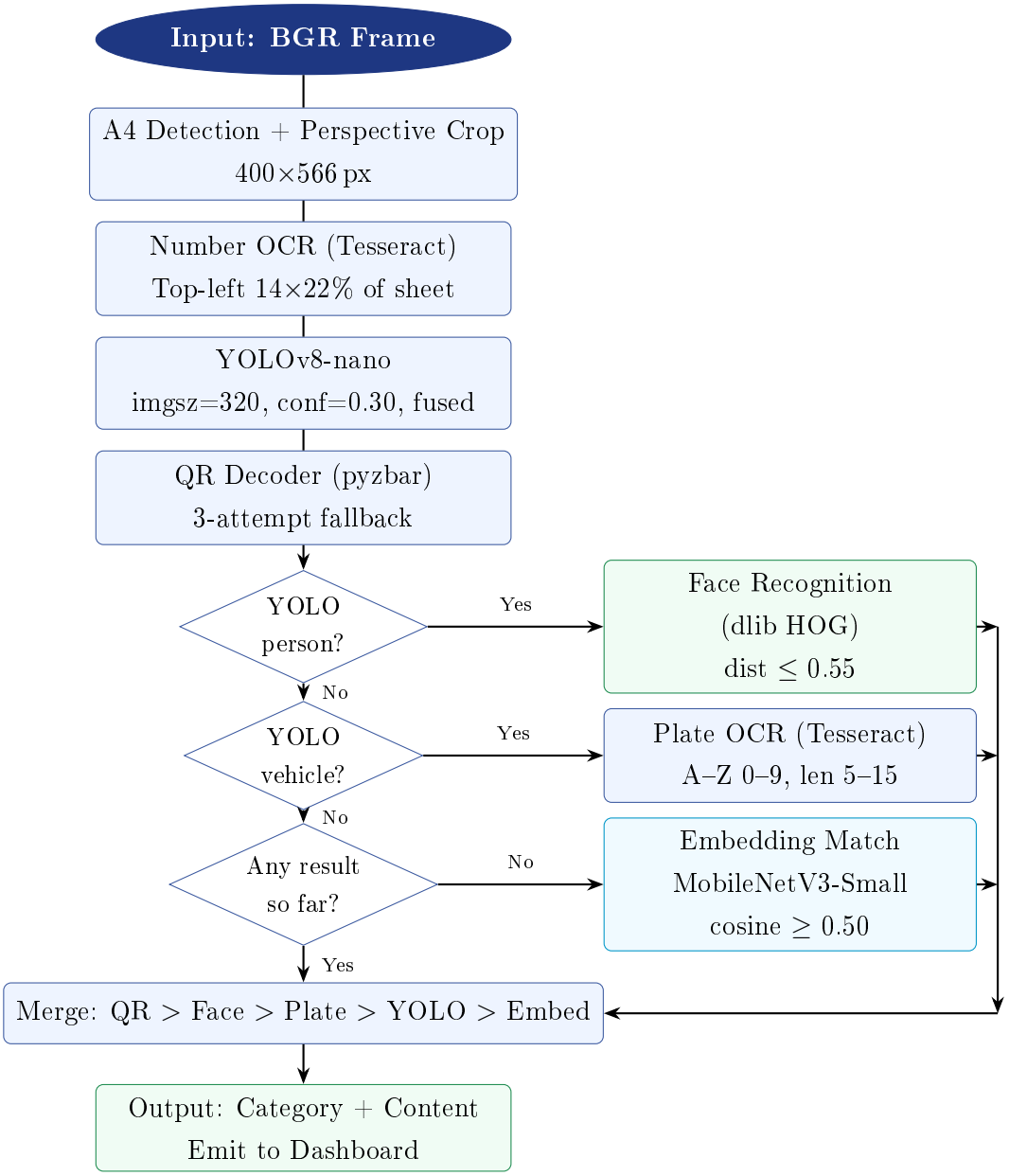

Figure 9. Five-model image recognition pipeline with conditional execution gates.

6.2 Detection Methods

6.2.1 YOLOv8-nano Object Detection

Table 8. YOLO COCO class mapping to competition categories

COCO Class	Competition Category	Competition Content
car	Vehicle	Car
truck	Vehicle	Truck
bus	Vehicle	Bus
bicycle	Vehicle	Bicycle
motorcycle	Vehicle	Motorbike
cat	Pets	Cat
dog	Pets	Dog
chair	Furniture	Chair
couch	Furniture	Sofa
bed	Furniture	Bed
dining table	Furniture	Dining Table
suitcase	Parcel	Parcel
person	(special)	Triggers face recognition pipeline

6.2.2 MobileNetV3-Small Embedding Matcher

A **576-dimensional embedding** is extracted and L2-normalised. Cosine similarity is computed against a pre-built reference database. Match threshold: ≥ 0.50. Fine-tuned weights (`mobilenet_finetuned.pth`) are loaded preferentially if present.

6.2.3 Face Recognition

Three identities in `face_encodings.pkl`: Henry Cavill, Keanu Reeves, Roger Federer. dlib HOG detector locates faces; 128-dim encodings compared at threshold ≤ 0.55. Images downscaled to max 500 px for fast detection.

6.2.4 QR Decoder

`pyzbar` attempts decoding on: original → grayscale → Otsu-thresholded. QR results carry confidence 1.0 and highest merge priority.

6.2.5 OCR Engine

Tesseract (≈70 ms) is primary; PaddleOCR (≈1200 ms) is fallback. Plate validation: contains both letters and digits, length 5–15, ≥80% alphanumeric.

6.3 Performance Summary

Table 9. Recognition pipeline performance on Raspberry Pi 5

Stage	Typical Time	Notes
A4 Detection	∼30 ms	Dual-method; every frame
Number OCR (Tesseract)	∼70 ms	Top-left region only (≈14% of sheet)
YOLOv8-nano	∼200 ms	imgsz=320; fused layers
QR Decode	<10 ms	Instant; every recognition call
Face Recognition	∼300 ms	HOG, downscaled; conditional on person
Plate OCR	∼100 ms	Tesseract; conditional on vehicle
Embedding Match	∼50 ms	DB dot-product; last resort only
Total (typical)	**700 ms–1.5 s**	Target: ≤ 2 s per card

7 Communication and Web Dashboard

7.1 Flask-SocketIO Backend

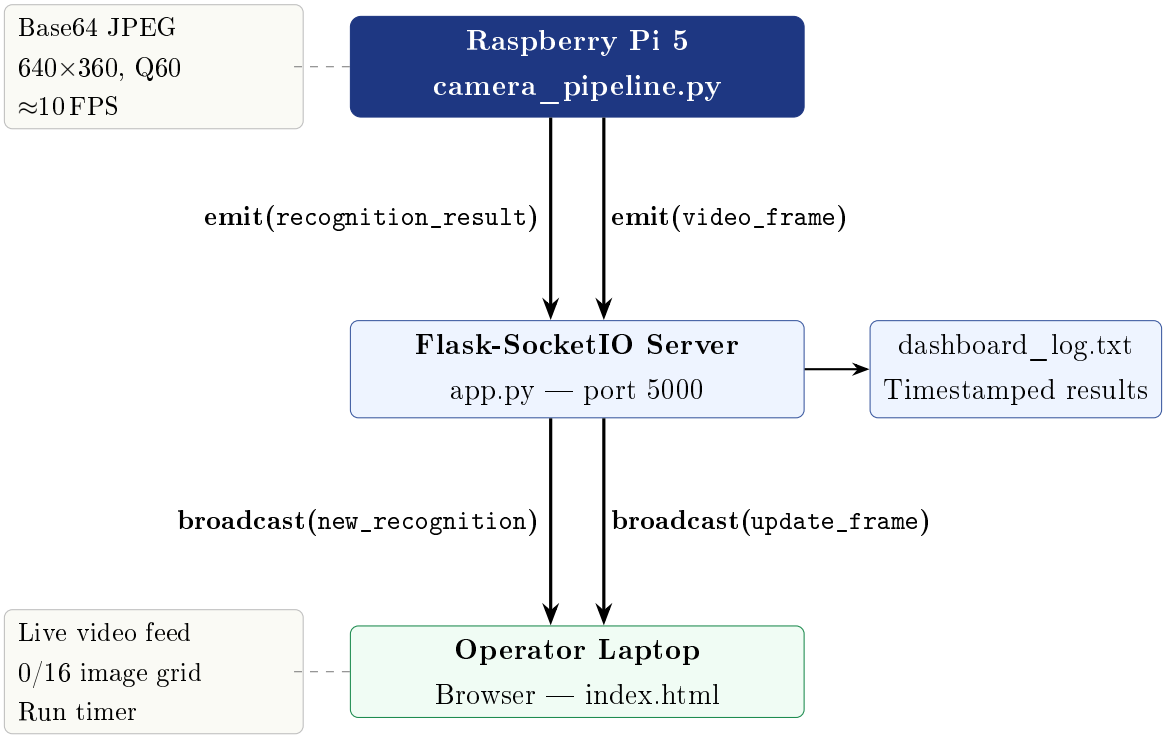

Figure 10. Flask-SocketIO event flow from camera pipeline to operator dashboard.

Key design decisions:

- `max_http_buffer_size=1e8` handles continuous base64 frame payloads without dropping.
- `cors_allowed_origins="*"` allows connection from any IP on the competition network.
- All recognition results are written to `dashboard_log.txt` with timestamps — verifiable by judges.
- Video compressed to 640×360 at JPEG quality 60, reducing bandwidth while maintaining clarity.

7.2 Frontend Dashboard Components

Table 10. Dashboard UI components and functions

Component	Location	Function
Live Camera Feed	Left panel, top	Base64 JPEG injected into `<img src>`; scanline animation; LIVE REC badge
System Logs	Left panel, bottom	Timestamped log lines; auto-scroll; system events and recognition results
Target Counter	Right panel, top	Updates dynamically as `X / 16` on each new recognition
Identified Targets	Right panel, body	3-column grid; each card shows index, cropped image, category, content
Run Timer	Header	MM:SS manual START/STOP; logs events; matches 5-min competition window
Connection LED	Header	Pulsing green = SocketIO connected; red = disconnected

8 Software Architecture

8.1 Repository Structure

Listing 1. Project repository structure

```
kriti2026-bot/
  firmware/
    RoboKriti.ino          # Arduino: line follow + obstacle
       avoidance
    ESP.ino                # ESP32: IMU task + radar task (FreeRTOS
       )
  vision/
    camera_pipeline.py     # Camera orchestrator + A4 detection +
       streaming
    recognize_mobilenet.py # Full 5-model ML recognition pipeline
    requirements.txt       # Pi vision dependencies
    db/                    # embeddings.pkl + face_encodings.pkl (
       offline)
  dashboard/
    app.py                 # Flask-SocketIO server (port 5000)
    templates/index.html   # Dashboard HTML structure
    static/script.js       # SocketIO client + UI logic
    static/style.css       # Dashboard styling
    mock_rover.py          # Test harness (no robot required)
    requirements.txt       # Dashboard dependencies
```

8.2 Software Dependencies

Table 11. Key software dependencies

Library	Purpose	Runtime
ultralytics	YOLOv8-nano object detection	RPi 5
torch / torchvision	Deep learning backend	RPi 5
transformers	CLIP image embeddings (HuggingFace)	RPi 5
facenet-pytorch	MTCNN face detection + InceptionResnetV1 recognition	RPi 5
pytesseract	Number OCR (A4 top-left region)	RPi 5
easyocr	Vehicle number plate OCR (scene text)	RPi 5
pyzbar	QR / barcode decoding	RPi 5
opencv-python-headless	Frame processing, A4 detection	RPi 5
picamera2	Pi Camera Module 3 driver	RPi 5
flask==3.0.1	Web server	RPi 5
flask-socketio==5.3.6	WebSocket event handling	RPi 5
python-socketio	SocketIO client (pipeline)	RPi 5
gpiozero	GPIO control abstraction	RPi 5
ESP32Servo	Servo PWM library	ESP32
MPU6050 (lib)	IMU I2C driver	ESP32
SoftwareSerial	Arduino UART emulation	Arduino

9 System Autonomy and Execution

9.1 Full Run Sequence

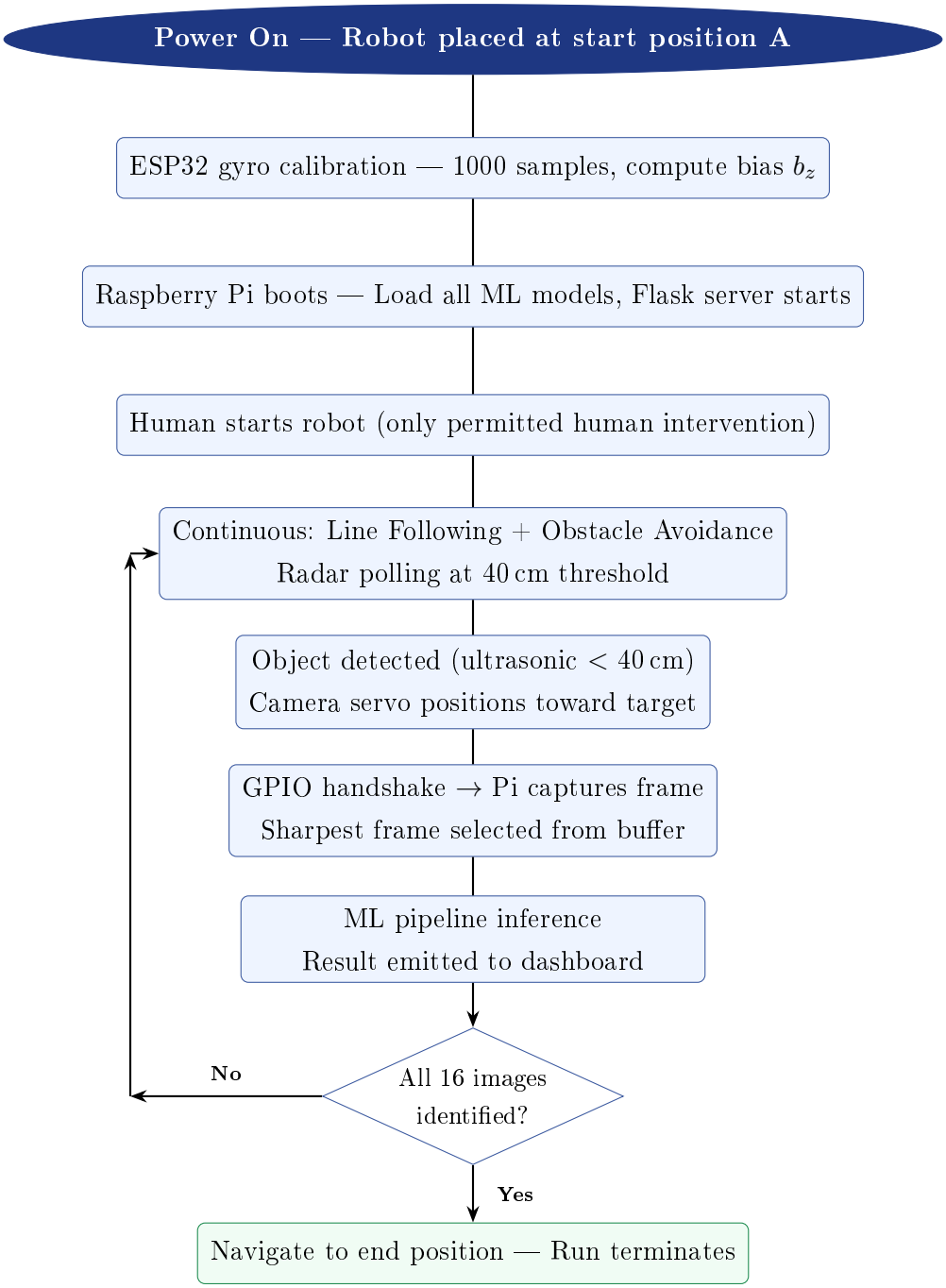

Figure 11. Complete autonomous run sequence from power-on to run termination.

9.2 Error Recovery Mechanisms

Table 12. Built-in error recovery mechanisms

Failure Mode	Recovery Mechanism
Line lost temporarily	75-cycle counter (≈1.5 s) before stop; `lastRight` guides re-acquisition
IMU read timeout	Returns −999 sentinel after 500 ms; turn continues with last valid yaw
Avoidance re-trigger	3-second cooldown prevents immediate re-detection after recovery
Camera servo jitter	100 ms debounce between servo repositions
Recognition exception	Worker thread exception-caught; returns error string without crashing
Dashboard disconnect	SocketIO auto-reconnects; log file provides offline fallback
Camera not found	Scans indices 0–3; falls back from Picamera2 to OpenCV

10 Testing and Validation

10.1 Firmware Testing

- Line following verified on 18–22 mm black tape; all 5 sensor states confirmed individually and in combination.
- IMU turn precision: 70° threshold consistently yields 88–92° physical rotation.
- Side and front obstacle avoidance tested with a 250 mm cube at various path positions.
- 3-second cooldown confirmed to prevent re-trigger oscillation post-recovery.

10.2 Vision System Testing

- A4 detection validated under normal indoor lighting; filters effectively reject false positives.
- Perspective correction produces consistent 400×566 px crops across varied viewing angles and distances.
- Full ML pipeline tested in `-test-folder` mode against representative image samples.
- `mock_rover.py` used to validate all dashboard behaviour without the physical robot.

10.3 Dashboard Testing

- SocketIO connection confirmed stable on local Wi-Fi at $\approx$10 FPS video.
- Recognition results correctly populate the 0/16 target grid.
- Timer and log events verified on START/STOP sequences.

11 Firmware Code Walkthrough

This section provides a concise step-by-step walkthrough of both firmware files — `RoboKriti.ino` (Arduino Uno) and `ESP.ino` (ESP32 DevModule) — explaining how each executes during a competition run.

11.1 Arduino Uno — `RoboKriti.ino`

1. **Initialisation (`setup()`).** All sensor pins (five line-following IR sensors, two side obstacle IRs, one front IR on ESP32 GPIO14), motor driver pins (L298N), and communication pins (`IMU_Reset`, `RadarScan`, SoftwareSerial on D2/D3) are configured. Both motor outputs are stopped and the control pins are driven LOW.
2. **Main loop — sensor read.** Each iteration reads all five line sensors (LL, L, M, R, RR) and prints their states to the hardware serial monitor for debugging.
3. **Priority-cascade line following.** The sensor states are evaluated in strict priority order: centre (M) → soft-left (L) → soft-right (R) → sharp-left (LL) → sharp-right (RR). The first matching condition sets the motor speeds for that cycle. LL/RR hits also update the `lastRight` junction flag.
4. **Lost-line handler.** If no sensor detects the line, a counter `f` increments on each loop tick. Once `f` reaches 160 ($\approx$3 s), the motors stop and await recovery.
5. **Obstacle cooldown gate.** All obstacle-avoidance logic is gated behind a 3 s cooldown (`millis() - lastAvoidTime > 3000`) to prevent re-trigger oscillation immediately after a recovery manoeuvre.
6. **Front obstacle avoidance (`obsAvoidFront`).** If the ESP32 sends an `"Obstacle Detected"` message over SoftwareSerial, the robot stops, then executes a three-turn U-bypass: rotate 90° in `lastRight` direction → forward 1 s → rotate 90° opposite → forward 2.3 s → rotate 90° opposite again → forward 0.5 s, then calls `MoveForwardTillLineSpecial()` to re-acquire the path.
7. **Side obstacle avoidance (`obsAvoid`).** If either side IR (`obsL` or `obsR`) goes LOW, the robot stops, reverses briefly for turning clearance, pulses `IMU_Reset` to zero the ESP32 yaw reference, calls `RotateTill90()` to rotate 90° using live gyroscope feedback, then calls `MoveForwardTillLine()` to advance until the outer line sensor detects the path, followed by a `FullScan()` radar trigger and path realignment.
8. **IMU-guided turning (`RotateTill90`).** A LOW→HIGH pulse on `IMU_Reset`

zeroes the ESP32 yaw reference. The Arduino then polls `ReadYaw()` over SoftwareSerial and drives the motors until $|\Delta\text{yaw}| \geq 70$; motor inertia completes the remaining ≈20°, yielding a physical 88–92° turn.

9. **Radar trigger (`FullScan`).** `RadarScan` is driven HIGH; the Arduino waits up to 5 s for a `"DONE"` acknowledgement from the ESP32 before driving it LOW again, ensuring a complete 0–180° ultrasonic sweep at every path re-entry point.

11.2 ESP32 DevModule — `ESP.ino`

1. **Initialisation (`setup()`).** UART2 (GPIO16/17) is opened at 9600 baud for Arduino communication. Both servo objects (`radarServo`, `cameraServo`) are attached. All ultrasonic trigger/echo pins (front/left/right HC-SR04s), front IR sensor (GPIO14), GPIO handshake pins (`toPI`, `fromPI`), and trigger input pins are configured. The MPU6050 is initialised over I^2C.

2. **Gyroscope bias calibration.** 1000 raw gyro-Z samples are collected at 2 ms intervals and averaged to compute `gyroBiasZ`, which is subtracted from every subsequent gyro reading to eliminate static drift.

3. **FreeRTOS task launch.** Two tasks are pinned to separate cores: the **IMU task** on Core 0 and the **Radar task** on Core 1, enabling true parallel execution.

4. **IMU task — Core 0 (`imu`).** Monitors `TRIGGER_PIN` (Arduino `IMU_Reset`) for a LOW→HIGH edge. On detection, `resetAll()` zeroes the angle accumulators and `startTracking()` enables continuous IMU integration. While tracking, `readIMU()` integrates gyro-Z with a complementary filter ($\alpha = 0.9$) and streams `"IMU:<angle>"` to the Arduino at ≈50 Hz. Tracking stops on a HIGH→LOW edge.

5. **Radar task — Core 1 (`radar`).** The radar servo oscillates between 30° and 150° in 5° steps with a 20 ms `vTaskDelay` per step. On each step:

   - If `TRIGGERC_PIN` (Arduino `RadarScan`) goes HIGH, a full 0–180° `sweep()` is executed and `"DONE"` is sent to the Arduino.
   - All three ultrasonic distances (front, left, right) are read.
   - If any distance is < 40 cm and the 200 ms camera cooldown has elapsed, the camera servo is pointed toward the detected side (0°, 90°, or 180°) and `PI_Signal()` triggers a Raspberry Pi image capture.
   - The front IR sensor is checked; a LOW reading sends `"Obstacle Detected"` to the Arduino over UART2.

6. **GPIO handshake with Raspberry Pi (`PI_Signal`).** `toPI` is driven HIGH; the

ESP32 waits up to 1 s for `fromPI` to go HIGH (acknowledgement from Pi), then drives `toPI` LOW. This ensures the Pi has confirmed the capture request before the radar task continues.

7. **Full sweep (`sweep`).** The radar servo sweeps 0°→180°→0° in 5° steps, printing any detected object angles (< 40 cm) to the serial monitor, then transmits `"DONE"` to the Arduino to release the `FullScan()` wait.

12 Conclusion

Team 2613 has designed and implemented a complete autonomous rover system addressing all nine evaluation parameters of the Kriti 2026 Bot Design Challenge. The three-tier architecture cleanly separates real-time navigation (Arduino Uno), parallel perception (ESP32 DevModule), and ML inference (Raspberry Pi 5), allowing each tier to operate at its optimal performance without blocking the others.

The system demonstrates robust engineering across the full stack: gyroscope-guided precision turning with startup bias calibration, dual-mode obstacle avoidance with state-aware path recovery, a five-model conditional ML ensemble targeting under 2 s inference per card, and a real-time web dashboard providing complete operator visibility during competition runs.

The governing design philosophy is **reliability over complexity**: conservative thresholds, cooldown timers, sensor fusion, and multi-level fallback chains ensure the robot behaves predictably and consistently across both competition runs.

A Key System Parameters

Table 13. Complete key parameter reference

Parameter	Value	Location
Motor PWM (normal speed)	255 (full)	RoboKriti.ino
IMU turn threshold	70°	RotateTill90()
IMU read timeout	500 ms	ReadYaw()
IMU data rate	50 Hz	ESP32 Core 0
Gyro calibration samples	1000	ESP32 setup()
Radar sweep range	30°– 150°	ESP32 Core 1
Radar step size	5°	ESP32 Core 1
Radar step delay	20 ms	vTaskDelay
Ultrasonic detection range	40 cm	ESP32 radar task
Camera servo cooldown	200 ms	ESP32
Pi GPIO handshake timeout	1000 ms	PI_Signal()
Avoidance cooldown	3000 ms	Arduino loop()
Lost-line patience	75 cycles (≈1.5 s)	Arduino loop()
Sharpness threshold	60.0 (Laplacian var.)	camera_pipeline.py
Frame buffer size	20 frames	camera_pipeline.py
A4 trigger cooldown	4.0 s	camera_pipeline.py
YOLO confidence threshold	0.30	recognize_mobilenet.py
YOLO inference image size	320 px	recognize_mobilenet.py
Embedding match threshold	0.50 cosine	recognize_mobilenet.py
Face match threshold	0.55 distance	recognize_mobilenet.py
Min plate text length	5 characters	recognize_mobilenet.py
Dashboard video FPS	≈10 FPS	camera_pipeline.py
Dashboard server port	5000	app.py

B Software Requirements Files

Vision system (Raspberry Pi 5):

```
ultralytics          # YOLOv8-nano
torch
torchvision
opencv-python-headless
Pillow
numpy
pyzbar               # QR decoding
transformers         # CLIP (CLIPModel, CLIPProcessor)
facenet-pytorch      # MTCNN + InceptionResnetV1 face recognition
pytesseract          # Number OCR (fast, clean A4 crop)
easyocr              # Vehicle number plate OCR (scene text)
# Legacy (backward compatibility)
face_recognition
open_clip_torch
paddleocr
paddlepaddle
python-socketio[client]
gpiozero
```

Dashboard server (Raspberry Pi 5):

```
flask==3.0.1
Flask-SocketIO==5.3.6
```